



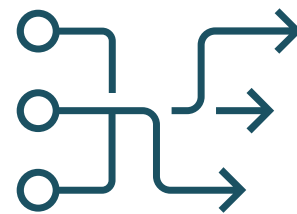
LECTURE

Introduction to RAG



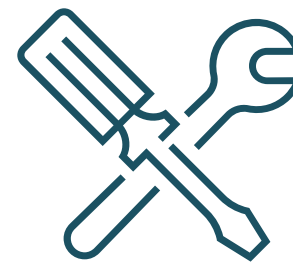
How do Language Models Learn Knowledge?

This will be the focus of this course



Model Pre-Training

- Training an LLM from scratch
- Requires large datasets (billions to trillions of tokens)



Model Fine Tuning

- Adapting a pre-trained LLM to specific data sets or domains
- Requires thousands of domain-specific or instruction examples



Passing Contextual Information

- Combining an LLM with external knowledge retrieval
- Requires external knowledge base
- How do we use vectors to **search** and provide **relevant context** to LLMs?

LESS Complexity and Compute-intensiveness



Passing Context to LMs Helps **Factual Recall**

- **Passing context** as model inputs improves factual recall
 - Analogy: Take an exam with **open notes**
- LLMs are evolving to accept a larger/infinite input token window size

Downsides of “Long” context:

- Higher API costs (# input token)
- Longer completion/inference times
- Content/documents in the middle may be overlooked (Lost in the middle and needle in haystack test)

⇒ Ongoing research to address “context limitation”



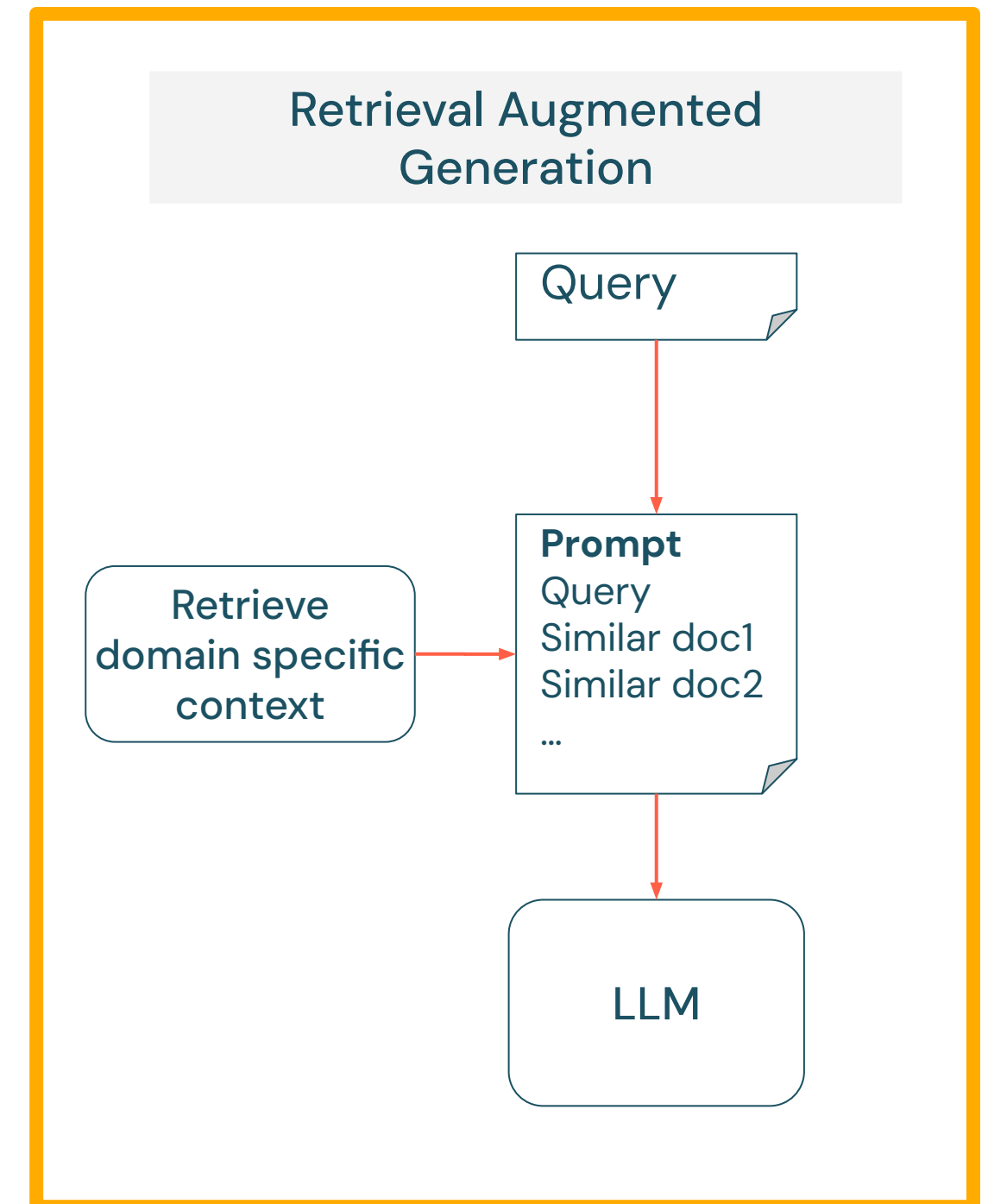
So what is RAG?



RAG is...

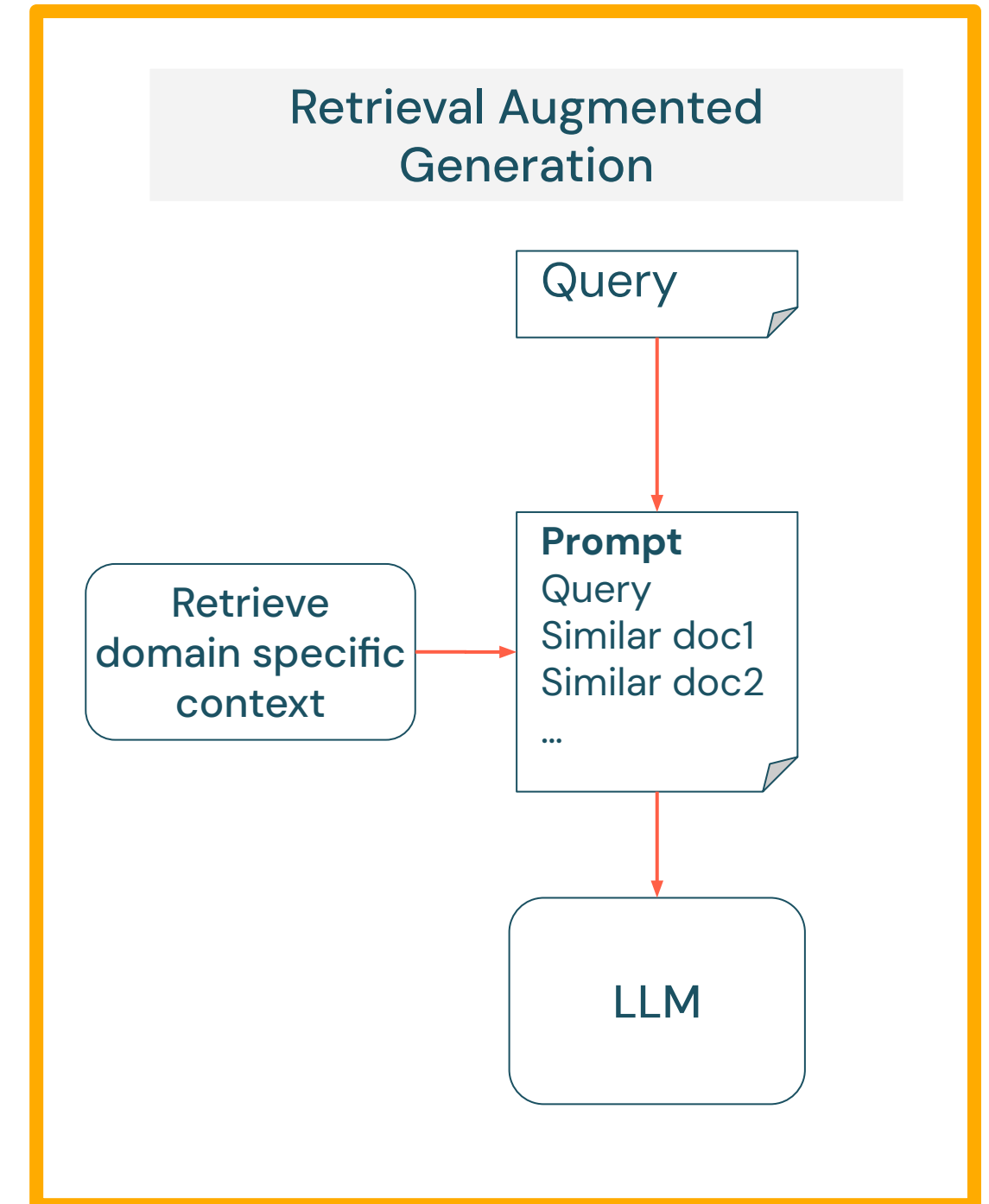
Retrieval Augmented Generation, or RAG;

- Is a pattern ([Lewis et al. 2020](#)) that can improve the **efficacy** of large language model (LLM) applications by **leveraging custom data**.
- Is done by **retrieving** data/documents relevant to a question or task and providing them as context to **augment** the prompts to an LLM to improve **generation**.



RAG is...

- The main problem that is solved with RAG architecture is the **knowledge gap**. This approach enhances the accuracy and relevance of responses.



RAG Use Cases

Some practical use cases

Q&A Chatbots

- Incorporate LLMs with chatbots to automatically derive more accurate answers.
- Used to automate customer support and website lead follow-up to answer questions and resolve issues quickly.

Search Augmentation

- Incorporating LLMs with search engines that augment search results with LLM-generated answers.
- Makes it easier for users to find the information they need.

Content Creation and Summarization

- Facilitate the development of high-quality articles, reports, and summaries using additional context.
- Example; generation of news articles or summarization of lengthy reports.



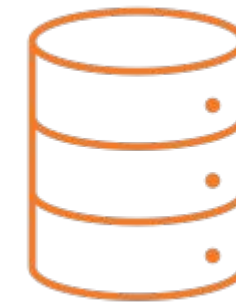
Main Concepts of RAG Workflow

Components and concepts in search and RAG workflow



Index & Embed:

An embedding model used for creating **vector representation** of the documents and user queries.



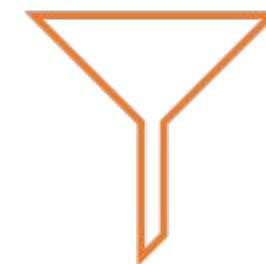
Vector Store:

Specialized to store unstructured data indexed by vectors. Vectors can be stored with a **vector DB, library, or plugin**.



Retrieval:

Search stored vectors using similarity search to efficiently **retrieve relevant information**.



Filtering & Reranking:

The process of selecting or ranking retrieved documents before passing as context.



Main Concepts of RAG Workflow

Components and concepts in search and RAG workflow



Prompt Augmentation:

Prompt engineering workflow to **enhance context via injection of data retrieved** from Vector store



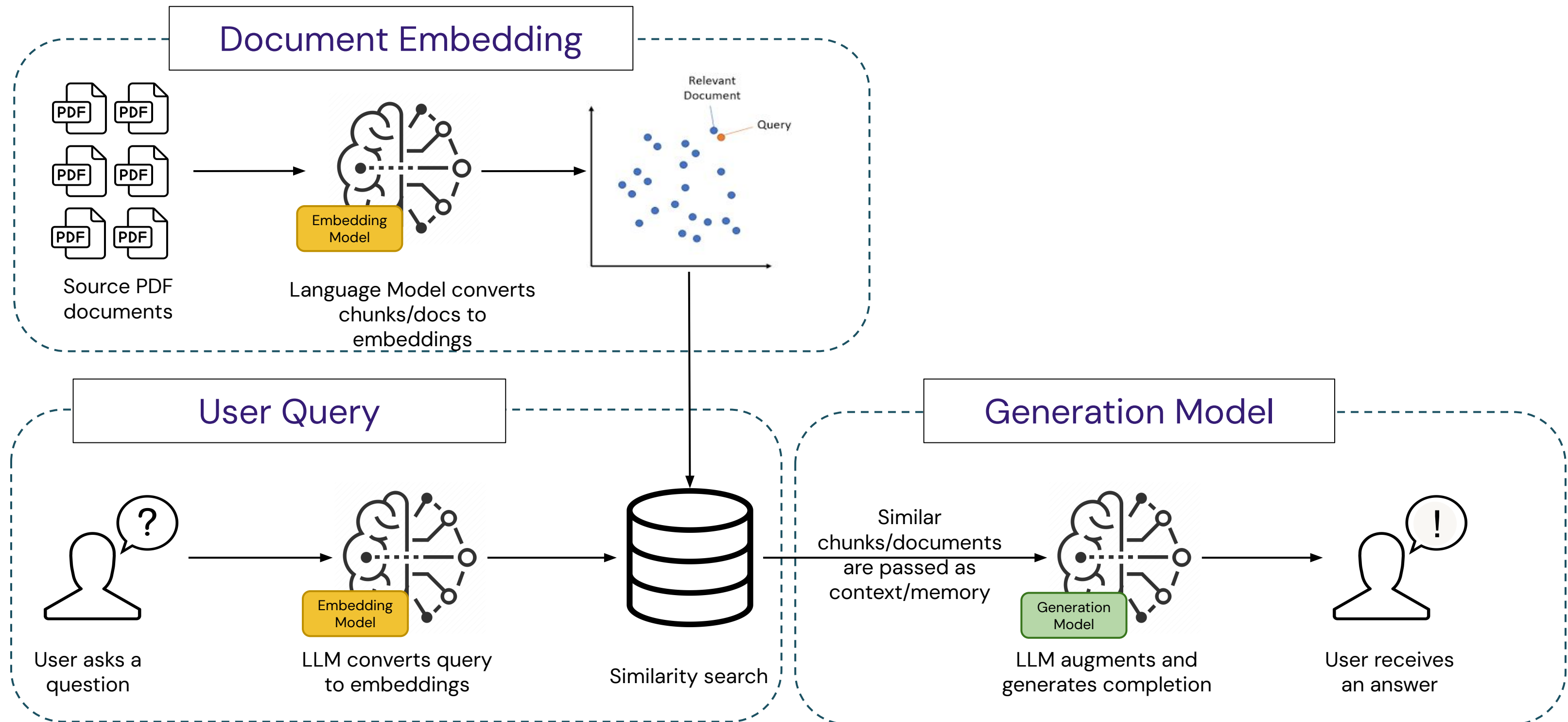
Generation:

A large language model used for generating a response for the user's request.



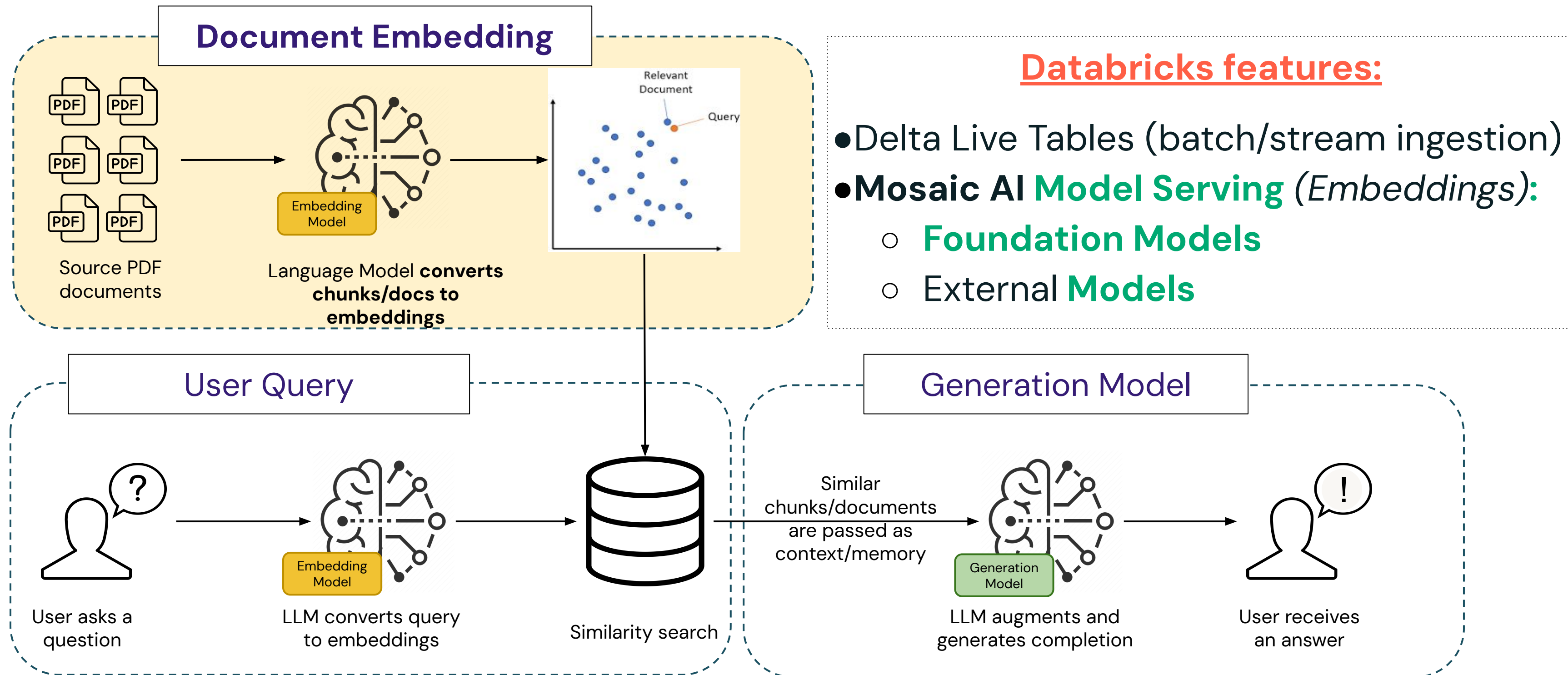
Retrieval Augmented Generation (RAG)

A sample architecture



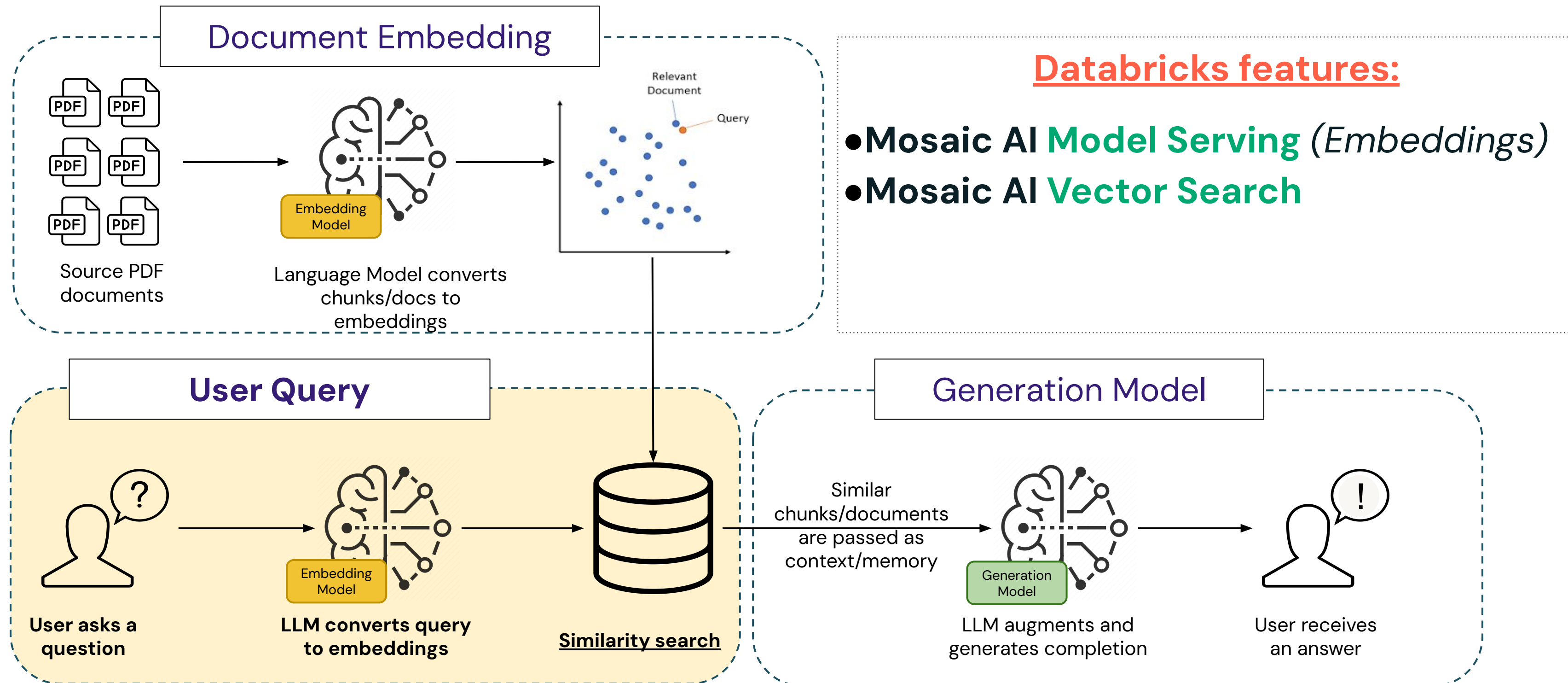
Retrieval Augmented Generation (RAG)

A sample architecture



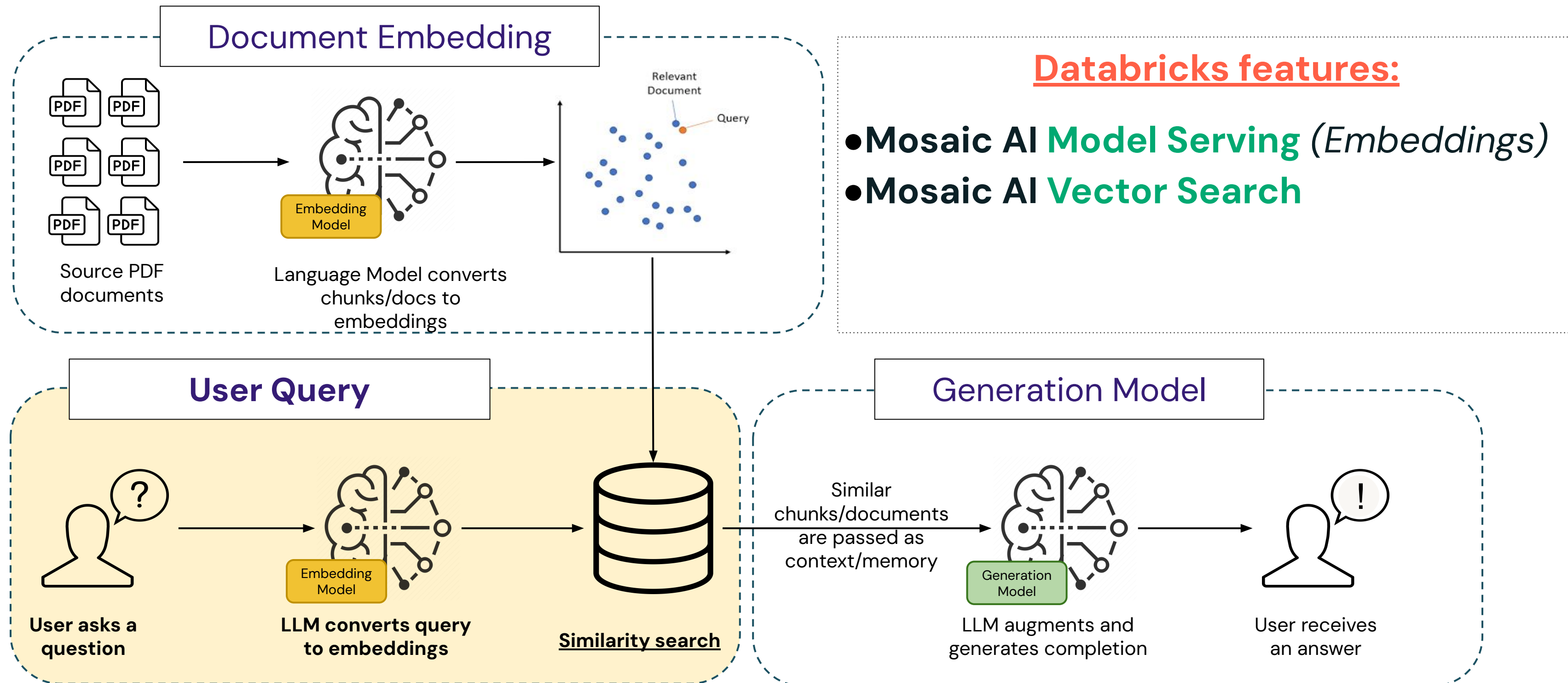
Retrieval Augmented Generation (RAG)

A sample architecture



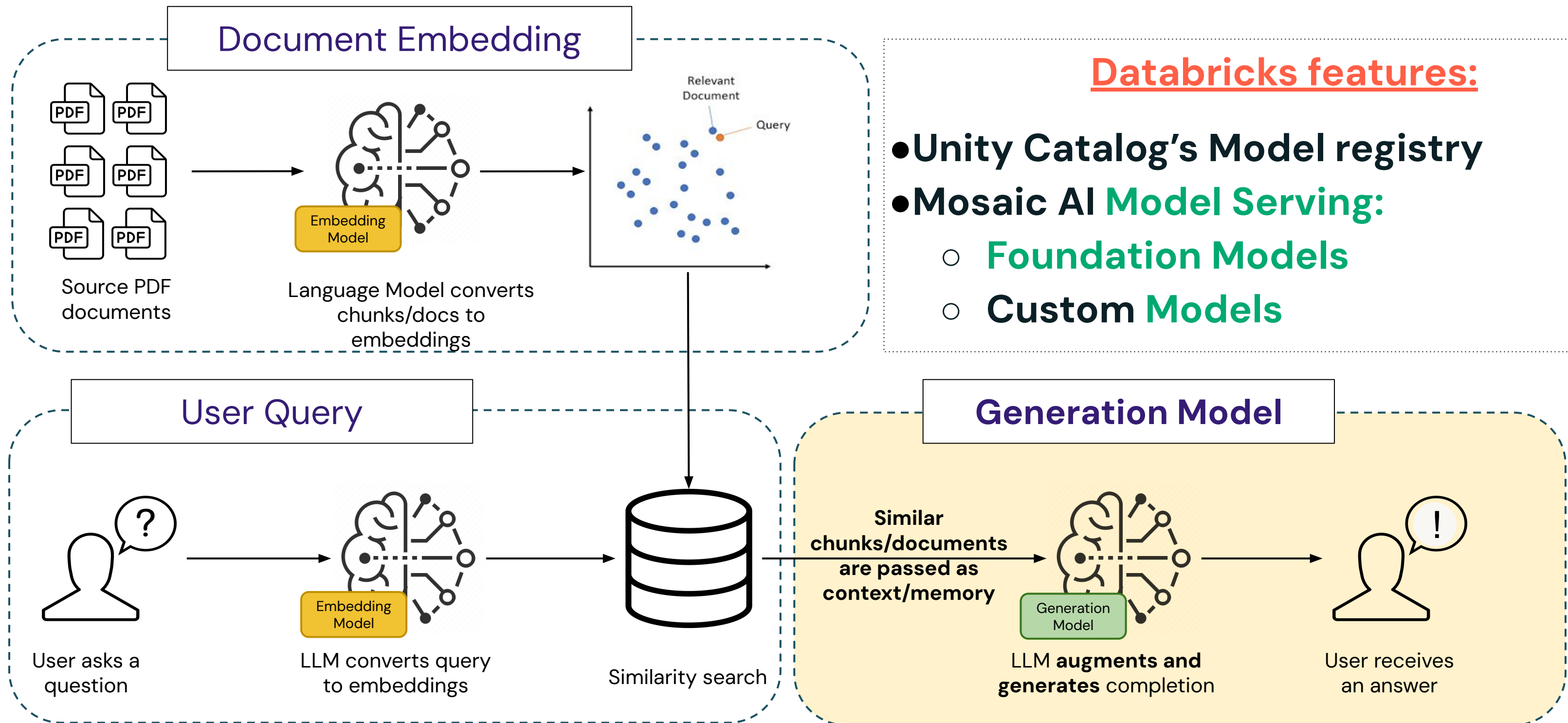
Retrieval Augmented Generation (RAG)

A sample architecture



Retrieval Augmented Generation (RAG)

A sample architecture



Benefits of RAG Architecture

Key benefits



Up-to-date and accurate responses

- LLM responses are not based solely on static training data.
- The model uses **up-to-date external data** sources to provide responses.



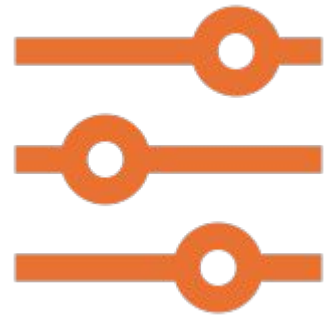
Reducing inaccurate responses, or hallucinations

- RAG attempts to mitigate the risk of producing “**hallucinations**” or incorrect information.
- Outputs can include citations of **original sources**, allowing human verification.



Benefits of RAG Architecture

Key benefits



Domain-specific contextualization

- Can be tailored to interface with **proprietary or domain-specific** data.
- Improves the accuracy of the response by using contextually relevant information.



Efficiency and cost-effectiveness

- Offers an alternative to fine-tuning LLMs by enabling in-context learning **without the up-front overhead of fine-tuning.**
- Beneficial where models need to be frequently updated with new data.



Mapping the RAG Workflow to Databricks Features

Find Relevant Context →

Mosaic AI **Vector Search**

- Built into the Databricks Intelligence Platform.
- Integrated with governance and productivity tools.
- Create and automatically sync with the underlying Delta table.

Generate Response →

Mosaic AI **Model Serving**

- State-of-the-art **foundation models** (e.g. Llama-3) and **external models**(e.g. GPT-4) made available by **Foundation Model APIs**.
- **Mosaic AI Playground:** Interact with supported models.

Serve RAG Chain →

Mosaic AI **Model Serving**

- Unified interface for deploying and querying models.
- Low-latency, high-availability model deployment service.
- Scales up or down to meet demand changes.

